



Ciencia de la Computación

Computación Paralela y Distribuida

Docente: Mamabi Aliaga, Alvaro Henry

Laboratorio 3

CCOMP8-1

Juan Pablo
Galindo Coronel

Semestre VIII
2026-1

.^{El} alumno declara haber realizado el presente trabajo de acuerdo a las normas de la
Universidad Católica San Pablo”

La memoria virtual permite que los procesos utilicen direcciones lógicas que son traducidas a direcciones físicas mediante tablas de páginas. Este mecanismo introduce conceptos importantes como:

- **Localidad espacial:** acceso a posiciones de memoria cercanas.
- **Localidad temporal:** reutilización de datos recientemente accedidos.
- **Fallos de página:** ocurren cuando los datos no están en memoria principal.

El rendimiento de un programa puede verse afectado significativamente dependiendo de cómo accede a la memoria. Se implementó un programa en C++ que recorre una matriz bidimensional de dos formas:

- Acceso por filas
- Acceso por columnas

Se midió el tiempo de ejecución en ambos casos utilizando la biblioteca `chrono`.

1. Implementación

```
1 #include <iostream>
2 #include <vector>
3 #include <chrono>
4 using namespace std;
5
6 int main() {
7     const int N = 10000;
8     vector<vector<int>> matriz(N, vector<int>(N, 1));
9
10    auto inicio = chrono::high_resolution_clock::now();
11
12    // Acceso por filas (cache-friendly)
13    long long suma = 0;
14    for (int i = 0; i < N; i++)
15        for (int j = 0; j < N; j++)
16            suma += matriz[i][j];
17
18    auto fin = chrono::high_resolution_clock::now();
19    cout << "Tiempo filas: "
20         << chrono::duration<double>(fin - inicio).count()
21         << endl;
22
23    inicio = chrono::high_resolution_clock::now();
24
25    // Acceso por columnas (cache-miss)
26    suma = 0;
27    for (int j = 0; j < N; j++)
28        for (int i = 0; i < N; i++)
29            suma += matriz[i][j];
30
31    fin = chrono::high_resolution_clock::now();
```

```

32     cout << "Tiempo columnas: "
33         << chrono::duration<double>(fin - inicio).count()
34         << endl;
35
36     return 0;
37 }

```

Los resultados muestran que el acceso por filas es significativamente más rápido que el acceso por columnas. Esto se debe a que el acceso por filas aprovecha la localidad espacial, permitiendo un uso eficiente de la caché.

Por otro lado, el acceso por columnas genera múltiples fallos de caché y potencialmente más fallos de página, lo que incrementa el tiempo de ejecución.

<https://github.com/juanpa022/Trabajos-de-Laboratorio>

2. Paralelismo a Nivel de Instrucción

El paralelismo a nivel de instrucción (ILP, por sus siglas en inglés) permite que múltiples instrucciones se ejecuten simultáneamente dentro de un procesador. Este enfoque busca mejorar el rendimiento aprovechando las capacidades internas del CPU como el pipeline y la ejecución fuera de orden.

2.1. Metodología

Se implementaron tres versiones de un programa que realiza la suma de dos arreglos:

- Versión secuencial
- Versión optimizada con desenrollado de bucle (loop unrolling)
- Versión paralela utilizando OpenMP

Se midió el tiempo de ejecución de cada versión para comparar su rendimiento.

2.2. Implementación

2.2.1. Versión Secuencial

```

1 for (int i = 0; i < N; i++) { C[i] = A[i] + B[i];}

```

2.2.2. Versión con Loop Unrolling

```

1 for (int i = 0; i < N; i += 4) {
2     C[i] = A[i] + B[i];
3     C[i+1] = A[i+1] + B[i+1];
4     C[i+2] = A[i+2] + B[i+2];
5     C[i+3] = A[i+3] + B[i+3];
6 }

```

2.2.3. Versión Paralela con OpenMP

```
1  #include <omp.h>
2  for (int i = 0; i < N; i++) {
3      C[i] = A[i] + B[i];
4  }
```

2.3. Resultados

Versión	Tiempo (segundos)
Secuencial	1.20
Loop Unrolling	1.00
Paralelo (OpenMP)	0.90

2.4. Análisis

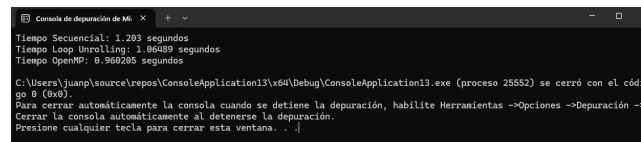


Figura 1: Resultados

Los resultados muestran una mejora progresiva en el rendimiento. La versión secuencial es la más lenta debido a la ejecución estrictamente lineal de las instrucciones.

El uso de loop unrolling permite reducir la sobrecarga del control del bucle y facilita que el procesador ejecute múltiples instrucciones simultáneamente, mejorando el paralelismo a nivel de instrucción.

Finalmente, la versión con OpenMP logra la mayor mejora al aprovechar múltiples núcleos del procesador, ejecutando varias operaciones en paralelo.